

Simulation-Based Inference with Generative Neural Networks

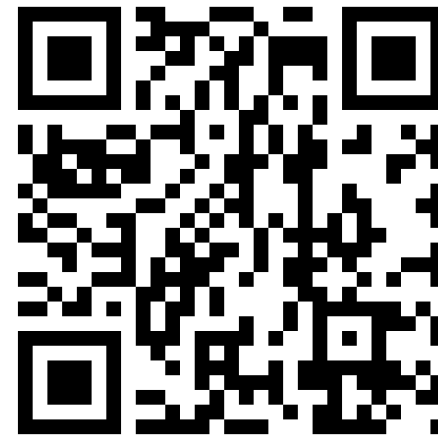
ProbAI School 2026

Jonas Arruda
University of Bonn

2026-08-06

Warm-Up

Join at
slido.com
#8530 715

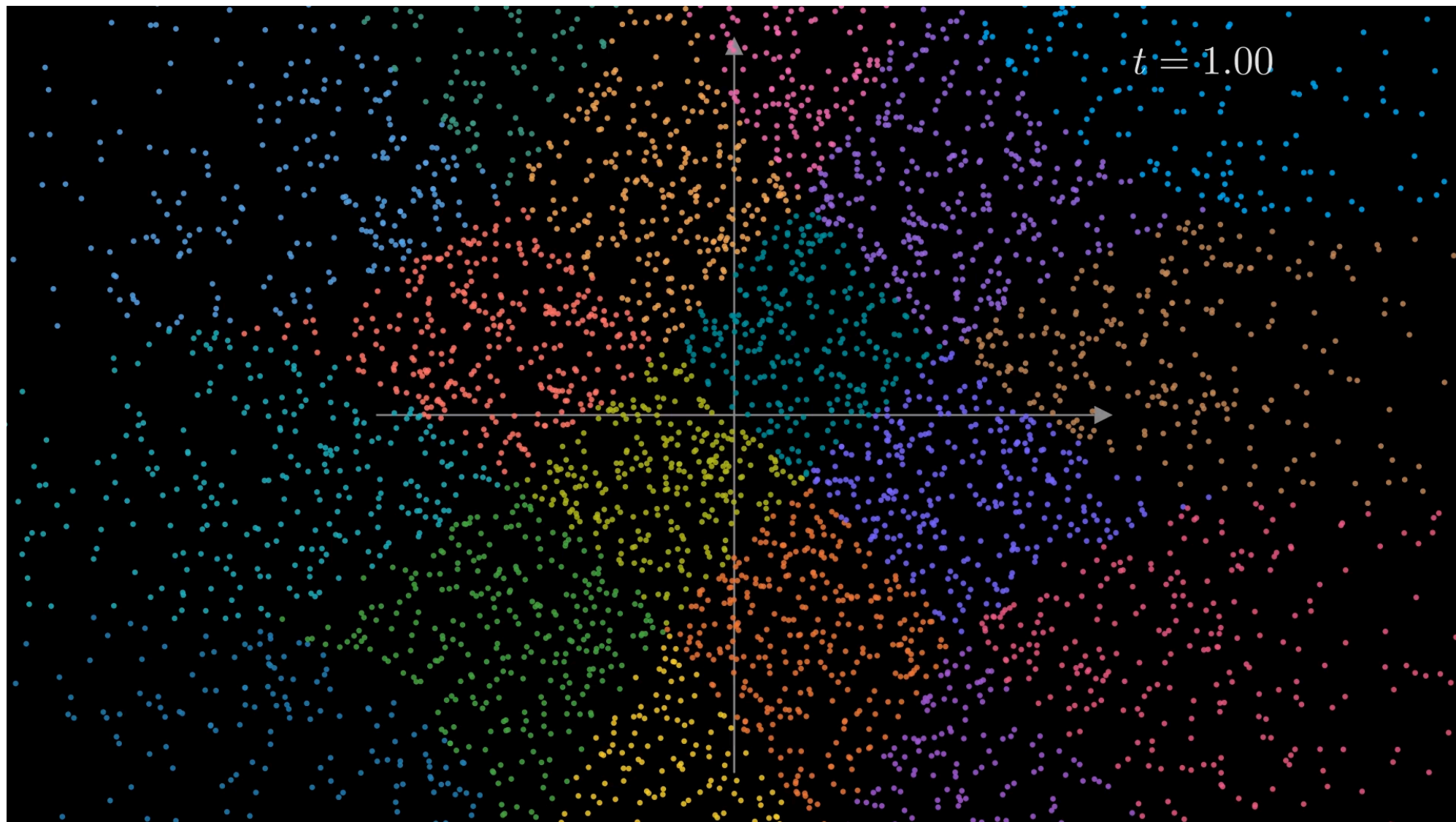


Overview

1. **What is SBI?** Introduction, Bayesian model types, normalizing flows.
→ *Exercise 1: from MCMC to amortized inference.*
2. **Did it work?** Calibration checking, model misspecification.
→ *Exercise 2: diagnostics on an epidemic time-series model.*
3. **How to leverage modern generative models?** Diffusion models, flow matching, consistency models.
→ *Exercise 3: diffusion models with post-hoc guidance.*

Part 1: What is Simulation-Based Inference?

Learning Densities

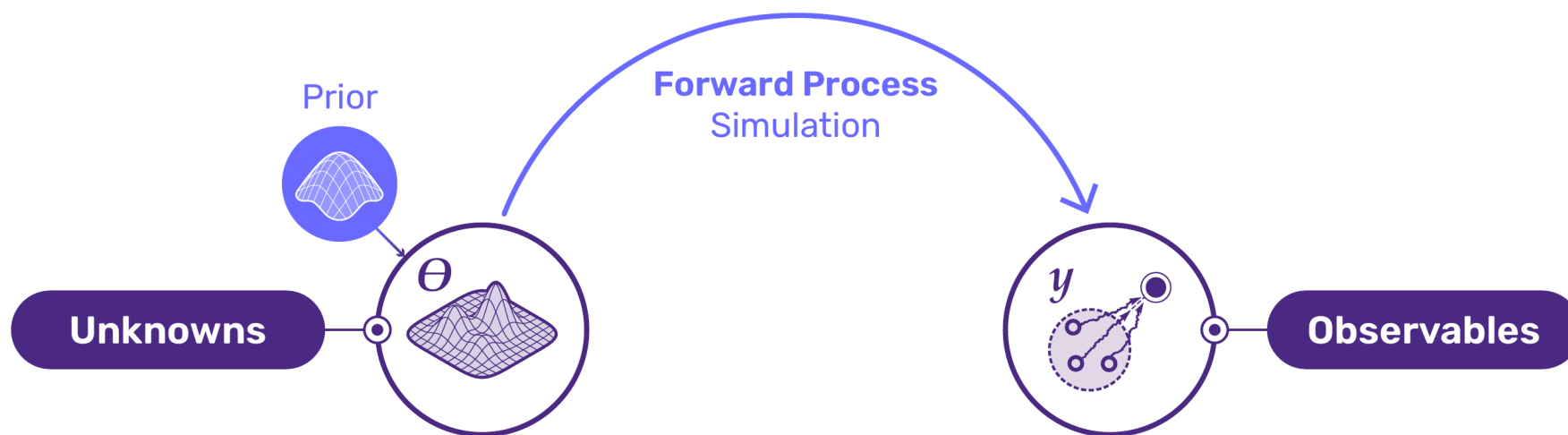


Everything today is about transporting a Gaussian onto our target, here a checkerboard.

Everything Starts with a Simulator

A **prior** and a scientific **simulator** together define the *forward process* from unknown parameters θ to observables y :

$$\theta \sim p(\theta), \quad y = \text{Sim}(\theta, u), \quad u \sim \text{RNG}(\cdot)$$

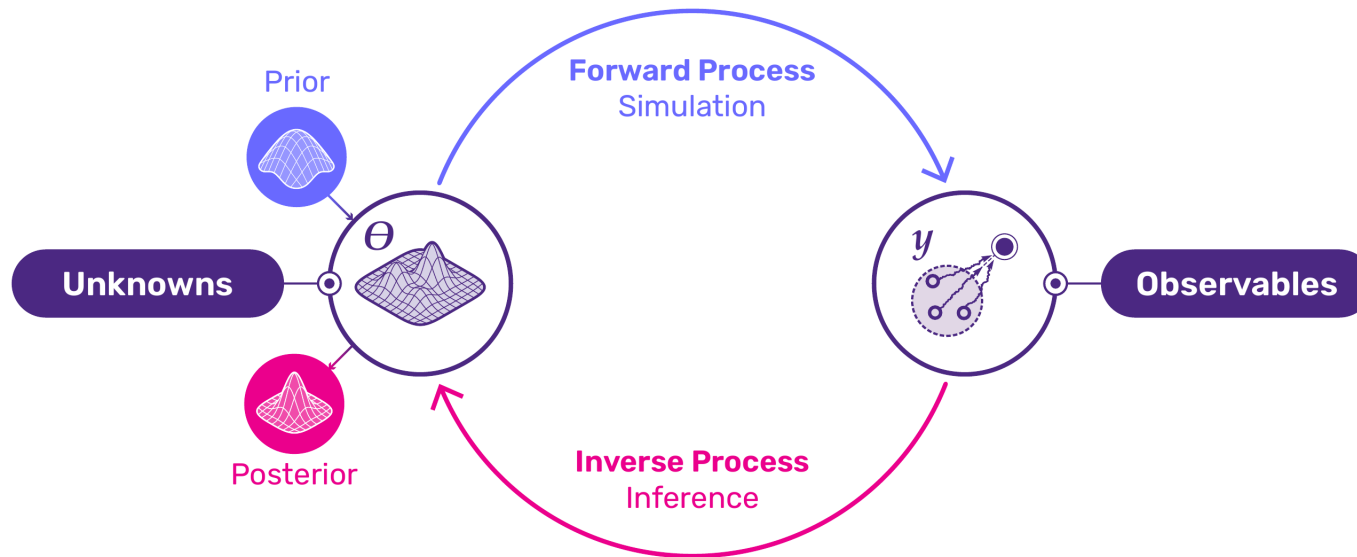


Parameters and data are drawn from the joint $\theta, y \sim p(\theta, y)$: our **Bayesian model**.

Figure: S. Radev, BayesFlow.

Forward and Inverse

- Running the simulator forward can be slow, but it is usually possible.
- What we actually want is the **inverse process**, the posterior $p(\theta \mid \mathbf{y}_{\text{obs}})$:



Likelihood-based (explicit)

- $p(\theta)$: sample and evaluate.
- $p(\mathbf{y} \mid \theta)$: sample and evaluate.

→ MCMC, VI (Day 1 & 2).

Simulation-based (implicit)

- $p(\theta)$: sample, evaluation *optional*.
- $p(\mathbf{y} \mid \theta)$: **sample only**, since the likelihood may be intractable.

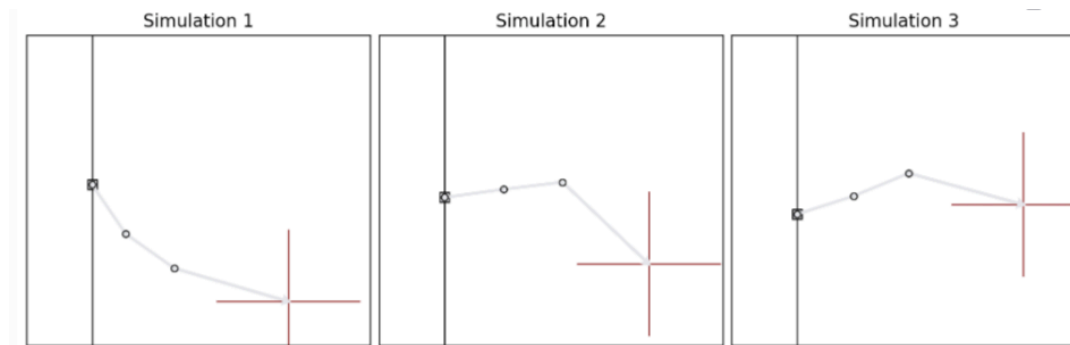
A Simple Toy Example

3-segment planar robot arm (Kruse et al., 2021)

- **Parameters** (not observed):

$$\theta = (h, \alpha_1, \alpha_2, \alpha_3)$$

- **Observation:** end position $\mathbf{y} = M(\theta)$.
- **Task:** recover $p(\theta \mid \mathbf{y})$.
- **Challenge:** different angles give the same observation $\mathbf{y}$, so the posterior is **multimodal**.



We will return to this example in Part 3.

Amortized Neural Posterior Estimation

Train a conditional density estimator $q_\phi(\boldsymbol{\theta} \mid \mathbf{y})$ on simulations $(\boldsymbol{\theta}, \mathbf{y})$ using the **forward KL**

$$\mathbb{E}_{p(\mathbf{y})} [\text{KL}(p(\boldsymbol{\theta} \mid \mathbf{y}) \parallel q_\phi(\boldsymbol{\theta} \mid \mathbf{y}))]$$

- The unique minimizer is the true posterior $q_\phi = p(\boldsymbol{\theta} \mid \mathbf{y})$.
- We can simplify this to:

$$\begin{aligned} \mathbb{E}_{p(\mathbf{y})} [\text{KL}(p(\boldsymbol{\theta} \mid \mathbf{y}) \parallel q_\phi(\boldsymbol{\theta} \mid \mathbf{y}))] &= \mathbb{E}_{p(\mathbf{y})} \mathbb{E}_{p(\boldsymbol{\theta} \mid \mathbf{y})} [\log p(\boldsymbol{\theta} \mid \mathbf{y}) - \log q_\phi(\boldsymbol{\theta} \mid \mathbf{y})] \\ &= \mathbb{E}_{(\boldsymbol{\theta}, \mathbf{y})} [\log p(\boldsymbol{\theta} \mid \mathbf{y}) - \log q_\phi(\boldsymbol{\theta} \mid \mathbf{y})] \\ &= \mathbb{E}_{(\boldsymbol{\theta}, \mathbf{y})} [\log p(\boldsymbol{\theta} \mid \mathbf{y})] - \mathbb{E}_{(\boldsymbol{\theta}, \mathbf{y})} [\log q_\phi(\boldsymbol{\theta} \mid \mathbf{y})]. \end{aligned}$$

- Minimizing allows us to drop the true posterior as it does not depend on ϕ :

$$\phi^* = \arg \min_{\phi} \mathbb{E}_{(\boldsymbol{\theta}, \mathbf{y}) \sim p(\boldsymbol{\theta}, \mathbf{y})} [-\log q_\phi(\boldsymbol{\theta} \mid \mathbf{y})]$$

Amortization means, we pay the training cost once and at inference on a new $\mathbf{y}_{\text{obs}}$ we only need a single evaluation of q_ϕ .

Normalizing Flows as Density Estimators

- How do we represent a flexible $q_\phi(\boldsymbol{\theta} \mid \mathbf{y})$ that we can both **sample** and **evaluate**?

→ With a neural network, but an invertible one!

- A **normalizing flow** transports a simple base $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ through a learned bijection $f_\phi(\cdot; \mathbf{y})$ (Papamakarios et al., 2021; Rezende and Mohamed, 2015):

$$\boldsymbol{\theta} = f_\phi^{-1}(\mathbf{z}; \mathbf{y}), \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- **Change of variables** gives a tractable density:

$$q_\phi(\boldsymbol{\theta} \mid \mathbf{y}) = p(f_\phi(\boldsymbol{\theta}; \mathbf{y})) \left| \det J_{f_\phi}(\boldsymbol{\theta}; \mathbf{y}) \right|$$

- Forward KL means minimize the log-likelihood of the flow.
- The neural network must be **invertible**, and we must be able to compute $\det J$.

Making Flows Cheap with Coupling Layers

- Computation of log-determinant costs $\mathcal{O}(d^3)$ in general
- But only $\mathcal{O}(d)$ if the Jacobian is **triangular**.

Coupling flow (Dinh et al., 2017): split $\theta = (\theta^A, \theta^B)$, transform one block conditioned on the other:

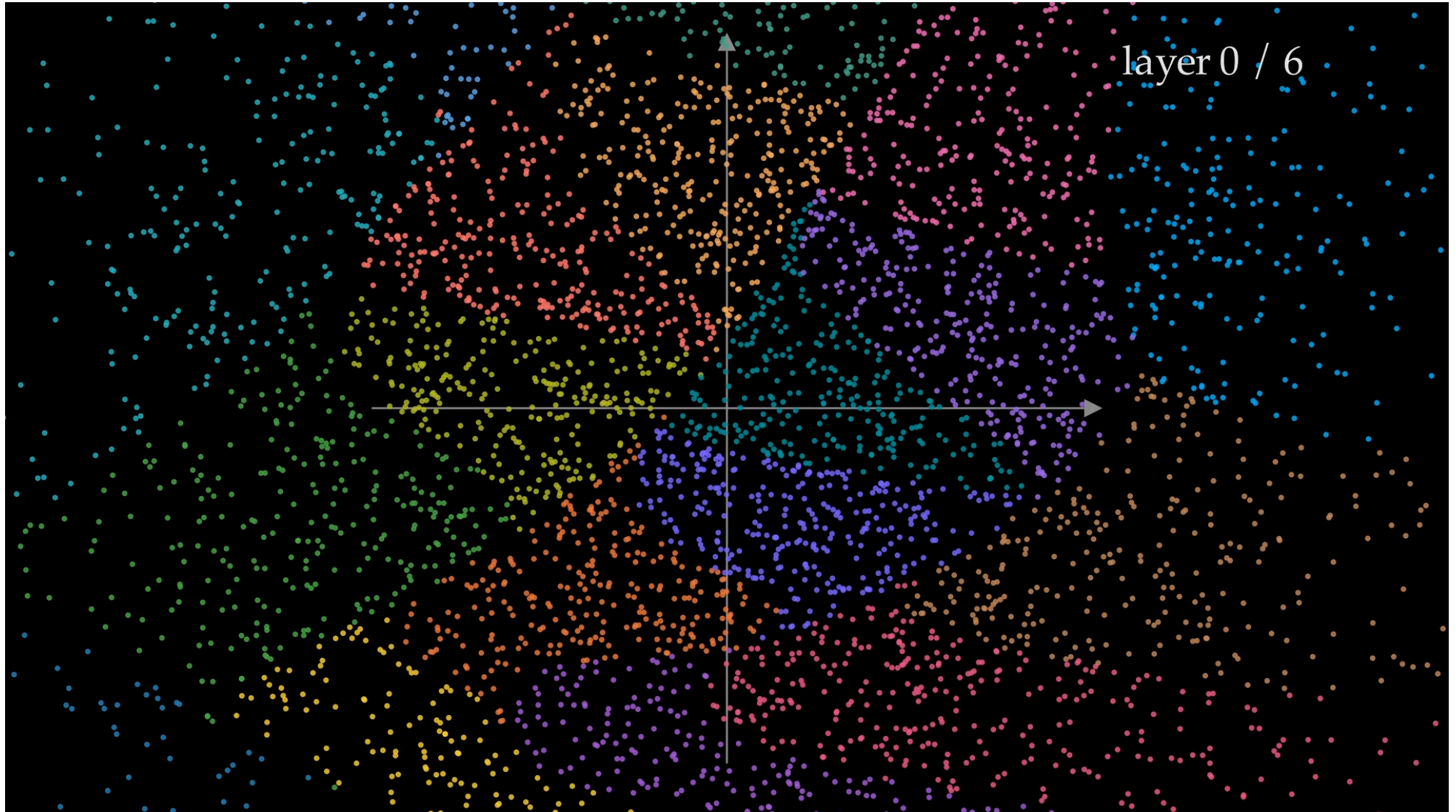
$$\mathbf{z}^A = f(\theta, \mathbf{y}) = \theta^A \odot \exp(s(\theta^B, \mathbf{y})) + t(\theta^B, \mathbf{y}), \quad \mathbf{z}^B = \theta^B$$

Then the inverse follows as:

$$\theta^A = (\mathbf{z}^A - t(\mathbf{z}^B, \mathbf{y})) \odot \exp(-s(\mathbf{z}^B, \mathbf{y})), \quad \theta^B = \mathbf{z}^B.$$

- s, t are **arbitrary neural nets** (they need not be invertible).
- The log-determinant is $\sum s(\cdot)$, and the inverse costs the same as the forward.
- Stack many layers $f = f^1 \circ \dots \circ f^L$, alternating which block is transformed.
- **Spline flows** (Durkan et al., 2019) use a more flexible transformation than affine layers.
- Stacked coupling blocks are **universal density approximators** for sufficiently regular densities (Draxler et al., 2024).

Normalizing Flows on a Checkerboard



Each coupling layer applies one invertible step.

Summary & Inference Networks

- Raw data is rarely a fixed-size vector.
- A summary network compresses it first, and both networks can be trained **jointly**:

$$\boldsymbol{\theta} \sim q_{\phi}(\boldsymbol{\theta} \mid \underbrace{s(\mathbf{y})}_{\text{summary}})$$

- **Summary network** s : compresses raw data $\mathbf{y}$ (set, series, hierarchy) into a fixed-size vector, using the inductive bias that matches the data.
- **Inference network** q_{ϕ} : the conditional network over parameters, taking $s(\mathbf{y})$ as its condition.
- Ideal case: s is a **sufficient statistic** for $\boldsymbol{\theta}$. In practice it is a *learned compression*, optimized for posterior inference (Radev et al., 2020).
- For plain vector data no summary network is needed: the flow conditions on $\mathbf{y}$ directly.

What Structure Does the Data Have?

The flow needs a **fixed-size** conditioning vector, and different data structures call for different summary networks:

Data type	Symmetry	Summary network
Exchangeable set	permutation invariance	DeepSet / Set Transformer
Time series	temporal order	GRU / CNN / Transformer
Hierarchical	nested groups	set-of-sets (composed)

Model Type 1: Exchangeable Data

- N i.i.d. observations: the posterior does **not** depend on ordering. What about **variable** N ?

$$s(\mathbf{y}_{\pi(1)}, \dots, \mathbf{y}_{\pi(N)}) = s(\mathbf{y}_1, \dots, \mathbf{y}_N) \quad \text{for all permutations } \pi$$

- **DeepSets** (Zaheer et al., 2017) enforce this by a *sum-decomposition*:

$$s(\mathbf{y}_1, \dots, \mathbf{y}_N) = \rho\left(\bigoplus_{i=1}^N \phi(\mathbf{y}_i)\right)$$

embed each element with ϕ , pool with a symmetric aggregator ($\sum$, mean, max), decode with ρ .

- **Set Transformer** (Lee et al., 2019): attention lets each element's representation depend on the whole set, which is more flexible but more expensive.

Model Type 2: Time Series

Ordering now carries information, so the summary network needs a **temporal** inductive bias.

Convolutional summaries share local filters across time:

$$s_{t,k} = \sigma \left(b_k + \sum_{l=-L}^L \sum_j w_{l,j,k} y_{t+l,j} \right)$$

Short-range patterns, position-invariant.

Transformers with time or positional embeddings also handle irregularly sampled series.

Recurrent (GRU/LSTM) summaries carry a hidden state:

$$\mathbf{h}_n = f_\phi(\mathbf{y}_n, \mathbf{h}_{n-1})$$

Long-range dependence via gating.

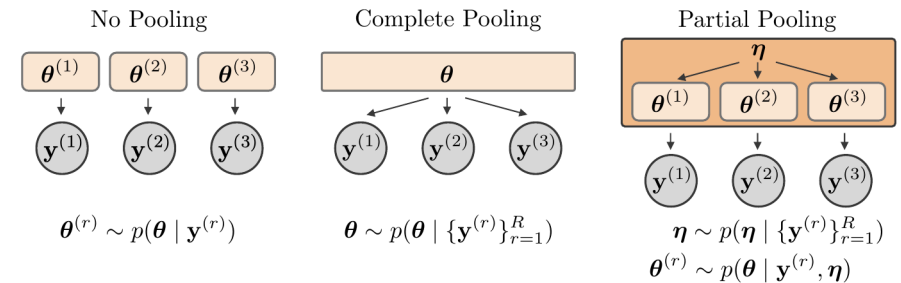
Model Type 3: Hierarchical Models

Data comes in **groups** (subjects, experiments, sites): local variation, shared global structure (Gelman et al., 2013).

$$\eta \sim p(\eta), \quad \theta^{(r)} \sim p(\theta \mid \eta), \quad \mathbf{y}^{(r)} \sim p(\mathbf{y} \mid \theta^{(r)})$$

Two coupled inference targets:

$$p(\eta \mid \{\mathbf{y}^{(r)}\}_{r=1}^R), \quad p(\theta \mid \mathbf{y}^{(r)}, \eta)$$



Summary: a set of sets. Encode each group, then aggregate across groups with a permutation-invariant network.

Bottleneck: naive amortization needs many simulator calls *per group*, so the budget grows with R . Part 3 comes back to this.

BayesFlow: Train Your Own Approximator

1 SIMULATE



Generate data from
any simulation you like

2 AMORTIZE



Part of the  Keras ecosystem
with **multi-backend support**

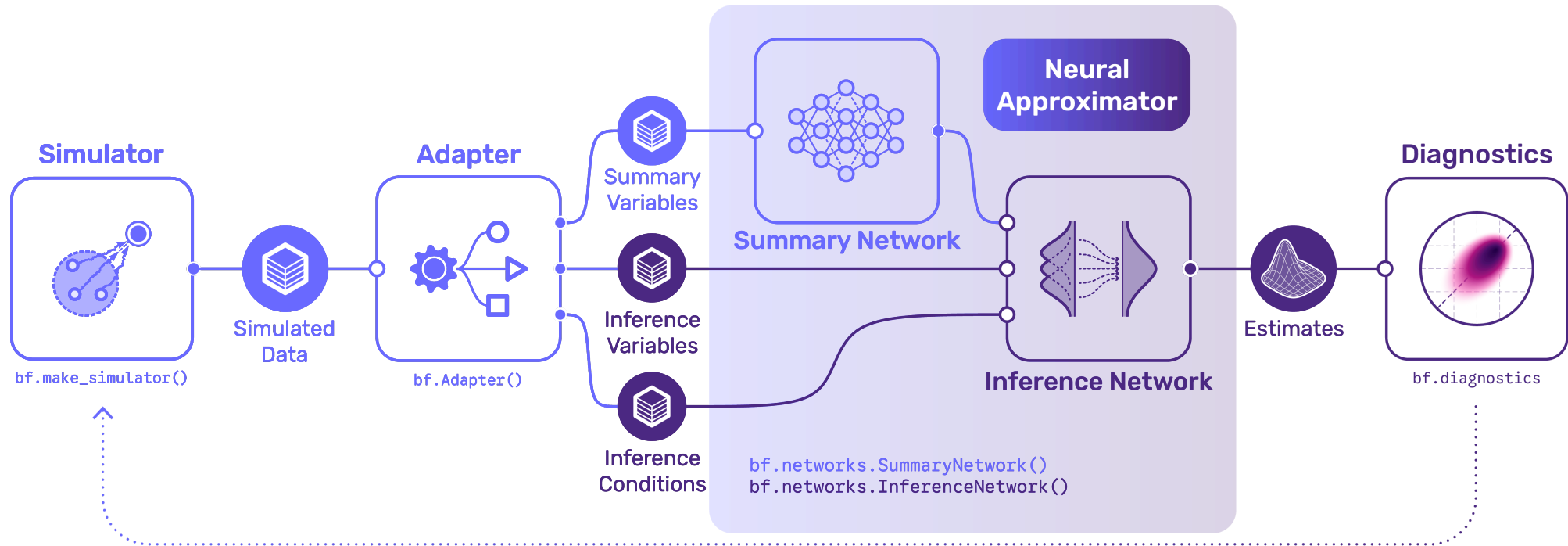
3 LEARN



Unleash simulation intelligence
with powerful **generative AI**

An open-source library for the full amortized Bayesian workflow: bayesflow.org

The Amortized Workflow



Simulator → **adapter** → **neural approximator** (summary + inference net) → **diagnostics**.

Figure: S. Radev, BayesFlow.

Code: A Basic Workflow

```
1 import bayesflow as bf
2
3 # 1. Simulator: draw (theta, y) pairs. The only thing you must provide.
4 simulator = bf.make_simulator([prior, likelihood_simulator])
5
6 # 2. Adapter: name/standardize/reshape variables for the networks
7 adapter = (bf.Adapter()
8            .standardize()
9            .concatenate(["beta", "sigma"], into="inference_variables")
10           .concatenate(["y"], into="inference_conditions"))
11
12 # 3. Networks: inference (a normalizing flow) + optional summary net
13 workflow = bf.BasicWorkflow(
14     simulator=simulator,
15     adapter=adapter,
16     inference_network=bf.networks.CouplingFlow(), # normalizing flow
17     summary_network=None, # vector data → none needed
18 )
```

Choosing a summary network:

```
1 summary_network=bf.networks.DeepSet() # exchangeable set
2 summary_network=bf.networks.TimeSeriesTransformer() # time series
```


Code: Train, then Query Any Dataset

```
1 # 4. Train on simulations (online: simulate fresh batches each step)
2 history = workflow.fit_online(epochs=50, batch_size=512)
3
4 # 5. Amortized inference: one forward pass per dataset
5 post = workflow.sample(conditions={"y": y_obs}, num_samples=2000)
6 #   → post["beta"]: (2000, d) posterior draws
7
8 # 500 new datasets, same network, no retraining:
9 post_many = workflow.sample(conditions={"y": Y_new_500}, num_samples=2000)
```

Exercise 1

From MCMC to Amortized Bayesian Inference

- Revisit a Bayesian **linear regression** in Pyro.
- Hand the *same* Pyro model to BayesFlow as a **simulator** and train an amortized posterior.
- Check that the BayesFlow posterior matches MCMC on that dataset.
- Then time inference on 500 further datasets, without retraining.



► Open in Colab 01_Linear_Regression at <https://shorturl.at/dx6PM>

Part 2: Does It Work?

Calibration & Misspecification

Two Distinct Questions

We produce an approximation $q(\boldsymbol{\theta} \mid \mathbf{y})$ of the true posterior. Before we trust it, we must ask:

1. **Is the inference faithful?** Does q recover the correct posterior *under the assumed model*?

→ **calibration checks** (SBC, TARP, C2ST).

2. **Is the model adequate?** Does the model explain the *observed* data at all?

→ **posterior/prior predictive checks, misspecification detection.**

- A posterior predictive check alone **cannot separate** the two (Schmitt et al., 2024).
- Amortization makes (1) cheap: re-running inference on thousands of simulated datasets costs a forward pass each.

Simulation-Based Calibration (SBC)

SBC exploits a **self-consistency** property of the Bayesian joint (Cook et al., 2006; Talts et al., 2018). Define

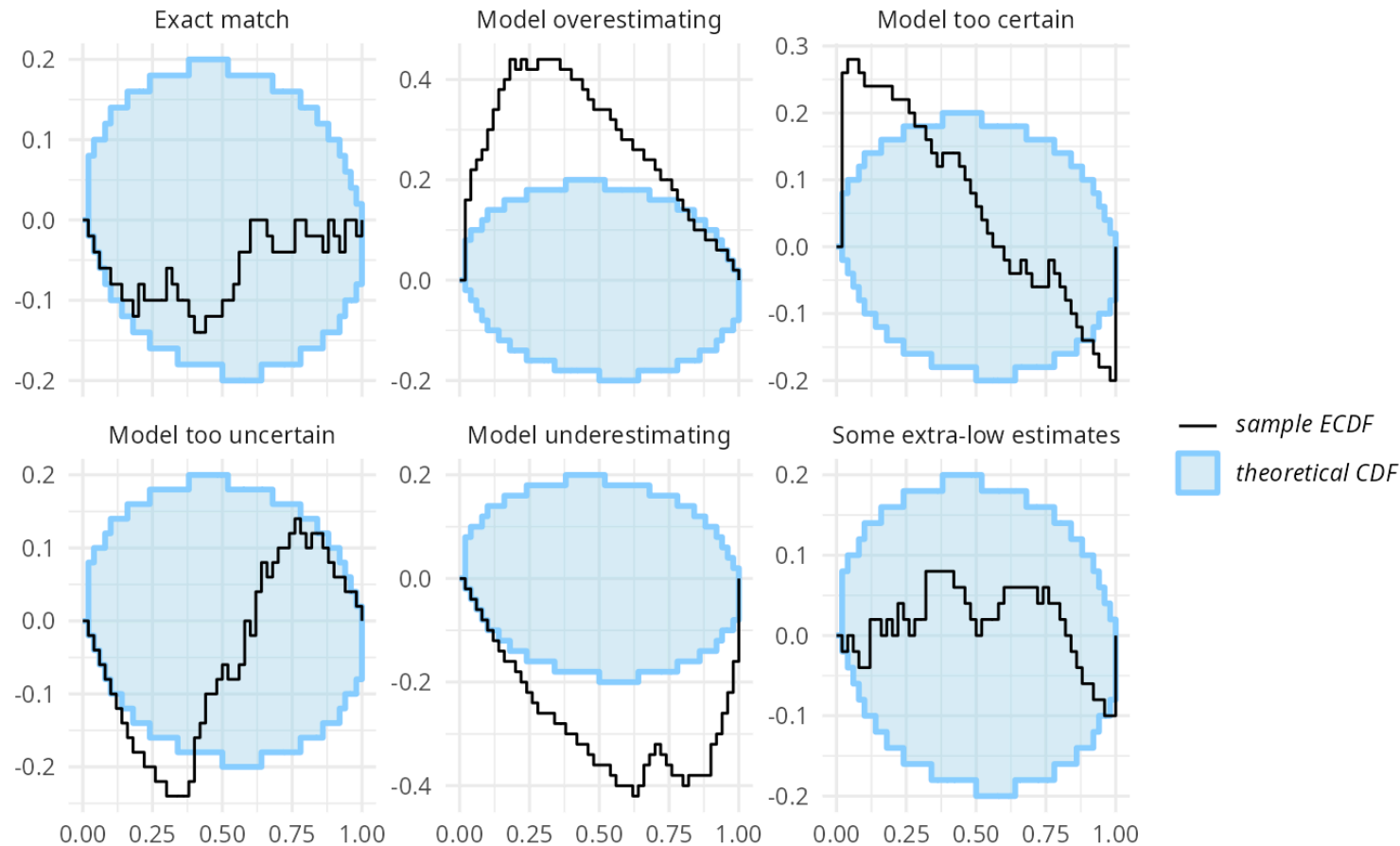
$$p_{\text{SBC}}(\mathbf{y}, \boldsymbol{\theta}, \tilde{\boldsymbol{\theta}}) = p(\boldsymbol{\theta}) p(\mathbf{y} \mid \boldsymbol{\theta}) q(\tilde{\boldsymbol{\theta}} \mid \mathbf{y}) = p(\mathbf{y}) p(\boldsymbol{\theta} \mid \mathbf{y}) q(\tilde{\boldsymbol{\theta}} \mid \mathbf{y})$$

If $q = p(\boldsymbol{\theta} \mid \mathbf{y})$, then $\boldsymbol{\theta}$ and $\tilde{\boldsymbol{\theta}}$ are **identically distributed** given $\mathbf{y}$.

Testing procedure: for many draws $(\boldsymbol{\theta}^{(r)}, \mathbf{y}^{(r)}) \sim p(\boldsymbol{\theta}, \mathbf{y})$, sample $\tilde{\boldsymbol{\theta}} \sim q(\cdot \mid \mathbf{y}^{(r)})$ and compute the **rank** of the true $\boldsymbol{\theta}^{(r)}$ among posterior draws.

Calibrated $\Rightarrow$ ranks are uniform.

Empirical Cumulative Distribution Functions as a Diagnostic Tool



Plot the **ECDF of ranks minus uniform**, with simultaneous confidence bands. Inside the band → calibrated. Image: [Martin Modrák](#).

SBC in Practice (BayesFlow)

```
1 # Simulate a fresh validation set the network never trained on
2 val = simulator.sample(1000)
3 post = workflow.sample(conditions=val, num_samples=500)
4
5 # Rank-ECDF calibration, per marginal
6 bf.diagnostics.plots.calibration_ecdf(
7     estimates=post, targets=val
8 )
9 # Recovery: posterior mean vs. ground truth
10 bf.diagnostics.plots.recovery(
11     estimates=post, targets=val
12 )
```

A Caveat: Marginal SBC Is Necessary, Not Sufficient

- Marginal rank checks can **pass** while the joint posterior is still wrong (Modrák et al., 2025).
- An estimator that ignores informative parts of the data may still show uniform marginal ranks, because errors can cancel across marginals.

Fix: use *data-dependent* test quantities $T(\boldsymbol{\theta}, \mathbf{y})$ so discrepancies accumulate instead of cancelling.

- **TARP** (Lemos et al., 2023): the distance to a random reference point, which depends on the data. Matching *all* references $\Leftrightarrow q = p$.
- **C2ST** (Lopez-Paz and Oquab, 2017; Yao and Domke, 2023): train a classifier to tell true joint $p(\boldsymbol{\theta}, \mathbf{y})$ from approximate joint $q(\boldsymbol{\theta} \mid \mathbf{y})p(\mathbf{y})$. Accuracy $\approx 0.5 \Rightarrow$ indistinguishable.

```

1 # TARP
2 bf.diagnostics.metrics.accuracy_random_points(
3     estimates=post, targets=val
4 )
5 # C2ST
6 bf.diagnostics.metrics.classifier_two_sample_test(
7     estimates=post[0], # for one dataset
8     targets=reference
9 )
10 # or concatenate posterior samples and data to test the joint

```


Model Misspecification

- Calibration assumes the model generated the data. Real simulators are imperfect:

$$\mathbf{y}_{\text{obs}} \approx p(\mathbf{y} \mid \boldsymbol{\theta}) \text{ for any } \boldsymbol{\theta}$$

- The network only ever saw the **prior predictive**.
- If $\mathbf{y}_{\text{obs}}$ falls outside that region it extrapolates and returns a confident but wrong posterior ([Frazier et al., 2024](#); [Schmitt et al., 2023](#)).
- Nothing raises an error: the estimator returns a posterior either way.
- The mismatch has to be **detected** explicitly.

Detecting Misspecification

- **Prior/posterior predictive check.** Simulate replicated data $\mathbf{y}^*$ from the fitted posterior; compare a test statistic T to $T(\mathbf{y}_{\text{obs}})$:

$$p_{\text{ppc}} = \Pr\{T(\mathbf{y}^*) \geq T(\mathbf{y}_{\text{obs}}) \mid \mathbf{y}_{\text{obs}}\}$$

- Extreme tail probability $\Rightarrow$ the model cannot reproduce the data.
- **Summary-space distance.** During training, learn the summary $s(\mathbf{y})$ under a distribution-matching penalty, for example MMD to $\mathcal{N}(\mathbf{0}, \mathbf{I})$ (Schmitt et al., 2023).
- At test time, an observation whose summary $s(\mathbf{y}_{\text{obs}})$ is an outlier in that space **flags** misspecification.
- The two checks answer different questions, so do both: calibration on simulations, and predictive adequacy on the real observation.

Exercise 2

Epidemic Time Series & Diagnostics

- Amortized posterior for a mechanistic **SIR** model (ODE + noisy reporting).
- A **recurrent (GRU)** summary network for the case-count time series.
- Run and interpret the diagnostic suite: SBC ECDF, recovery, contraction.
- Then confront the network with **real COVID-19 data** and judge model adequacy.



► Open in Colab 02_SIR_Diagnostics at <https://shorturl.at/7Vdl6>

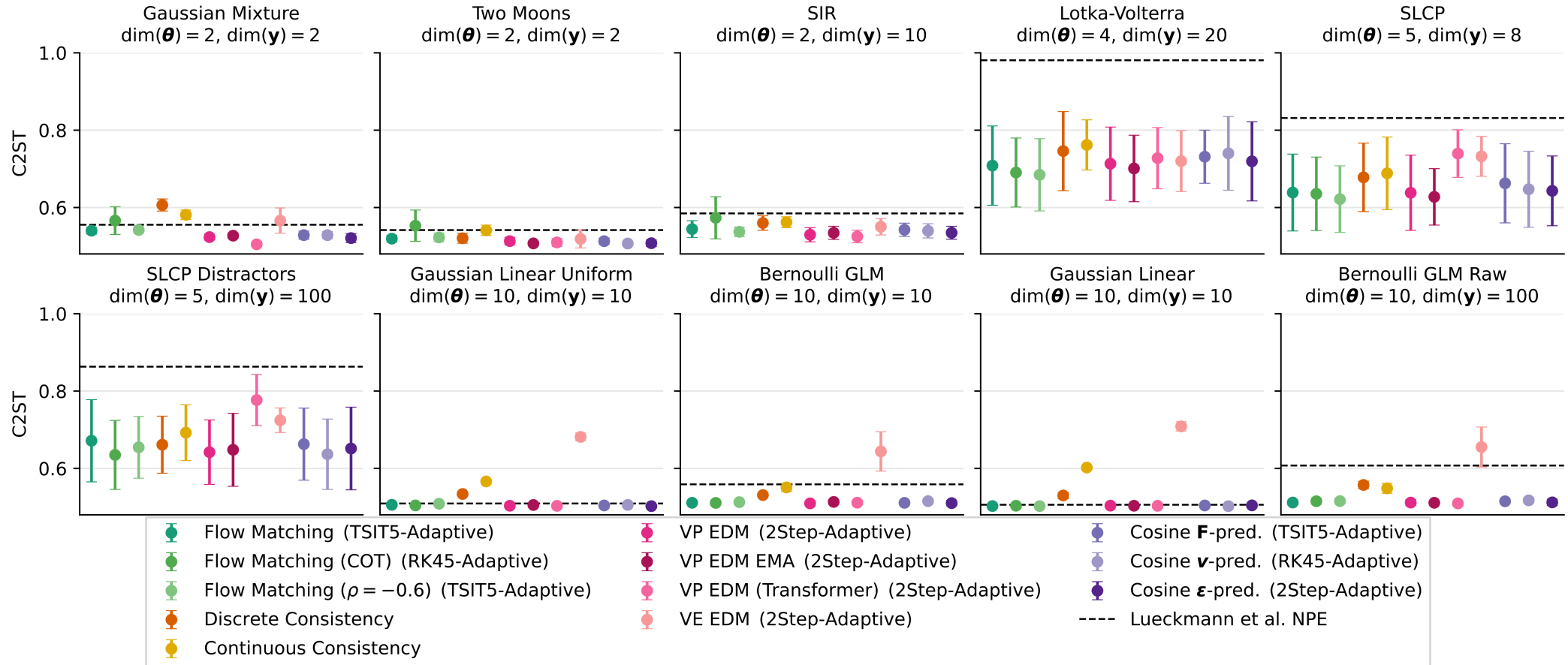
Part 3: Diffusion Models, Flow Matching, and Consistency Models

Why Go Beyond Normalizing Flows?

Family	Architecture	Sampling	Density
Normalizing Flows	constrained (invertible)	1 step	fast
Diffusion Models	free-form	multi-step	slow
Flow Matching	free-form	multi-step	slow
Consistency Models	free-form	few-step	N/A

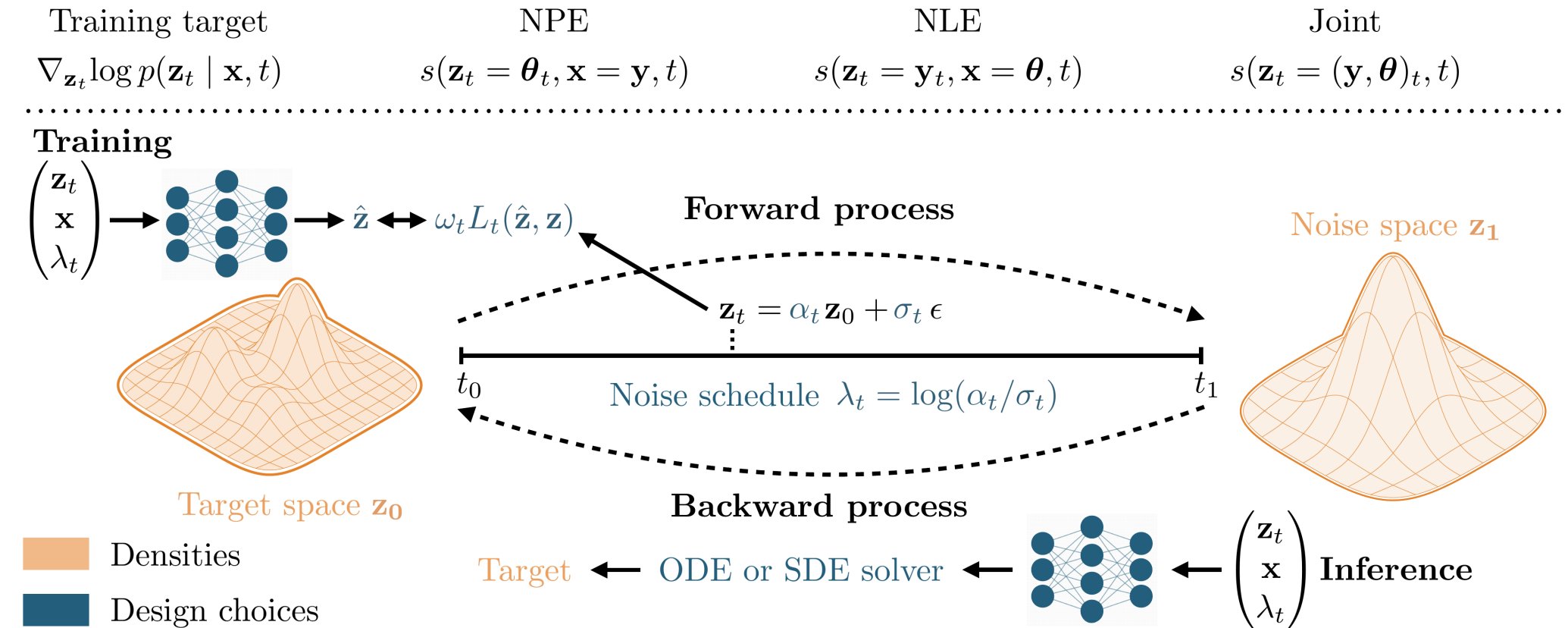
- Dropping invertibility frees the architecture.
- What we gain is expressivity and the ability to **steer sampling after training**.
- We pay with lower sampling speed and, for consistency models, the density.

Benchmarking Diffusion Models



C2ST across ten benchmark tasks, so lower is better and 0.5 means indistinguishable from the reference posterior. The dashed line is the normalizing flow baseline of Lueckmann et al. (Arruda et al., 2025).

Diffusion Models



Forward noising ($\mathbf{z}_t = \alpha_t \mathbf{z}_0 + \sigma_t \epsilon$) → train the network on $\omega_t L_t(\hat{\mathbf{z}}, \mathbf{z})$ → backward via an **ODE/SDE solver** (Arruda et al., 2025).

Diffusion: The Forward Process

Start at the target $\mathbf{z}_0 = \boldsymbol{\theta}$. Gradually add noise via an SDE (Song et al., 2021):

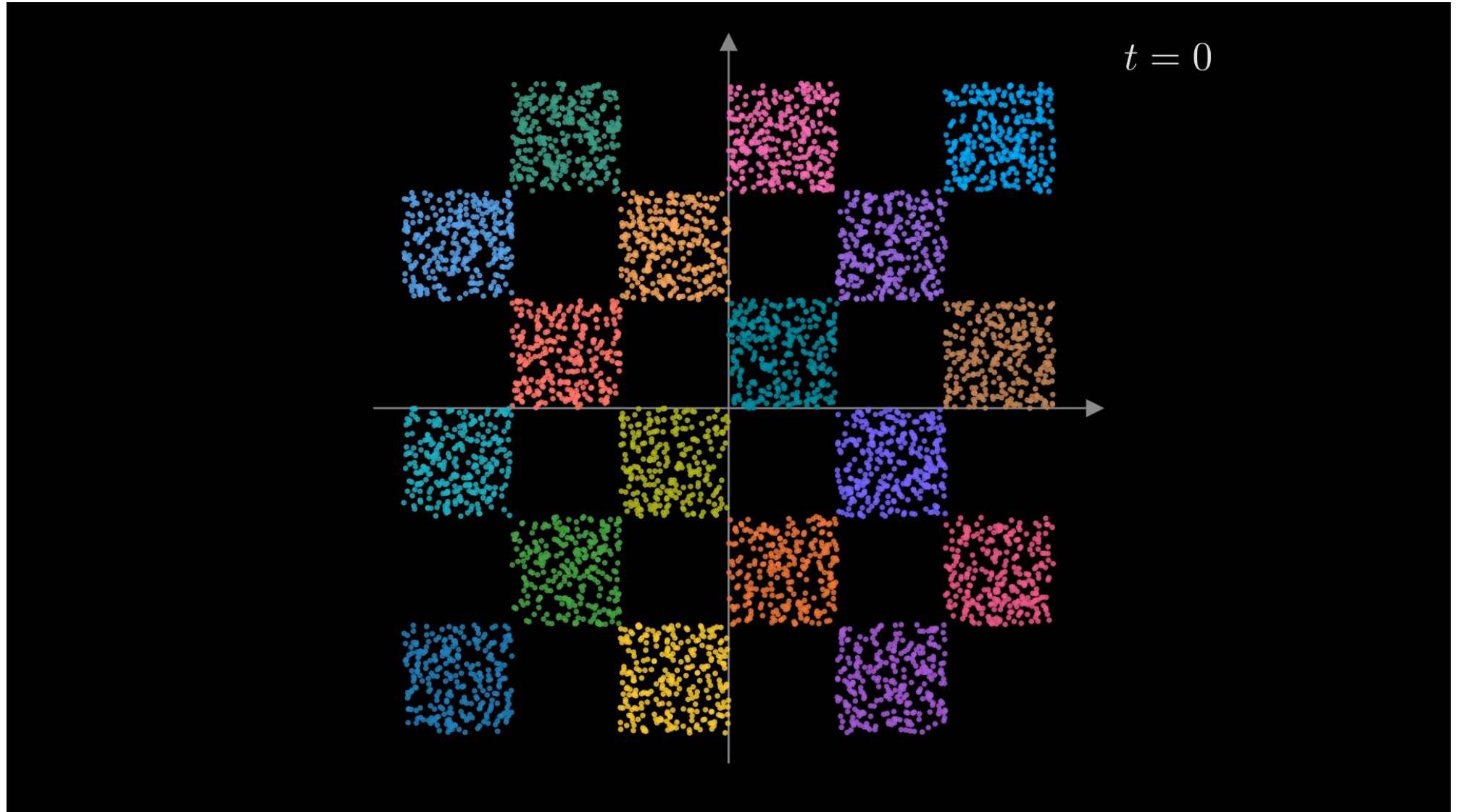
$$d\mathbf{z}_t = f(t) \mathbf{z}_t dt + g(t) d\mathbf{W}_t$$

The forward transition is **Gaussian in closed form**, so nothing has to be simulated:

$$p(\mathbf{z}_t \mid \mathbf{z}_0) = \mathcal{N}(\alpha_t \mathbf{z}_0, \sigma_t^2 \mathbf{I}) \quad \Longleftrightarrow \quad \mathbf{z}_t = \alpha_t \mathbf{z}_0 + \sigma_t \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

We can jump to any noise level t directly.

Forward Process on a Checkerboard



A checkerboard target $\mathbf{z}_0$ melts into isotropic Gaussian noise as $t : 0 \rightarrow 1$.

Jonas Arruda · University of Bonn · ProbAI School 2026

Diffusion: The Reverse Process

- The forward SDE has a **time-reversal** (Anderson, 1982):

$$d\mathbf{z}_t = \left[f(t)\mathbf{z}_t - g(t)^2 \nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t) \right] dt + g(t) d\bar{\mathbf{W}}_t$$

- The only unknown is the **score** $\nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t)$.
- If known, we can integrate backwards from noise to a sample.

Denoising Score Matching

- Regress a network $\hat{s}$ onto the score of the (Gaussian) noising kernel:

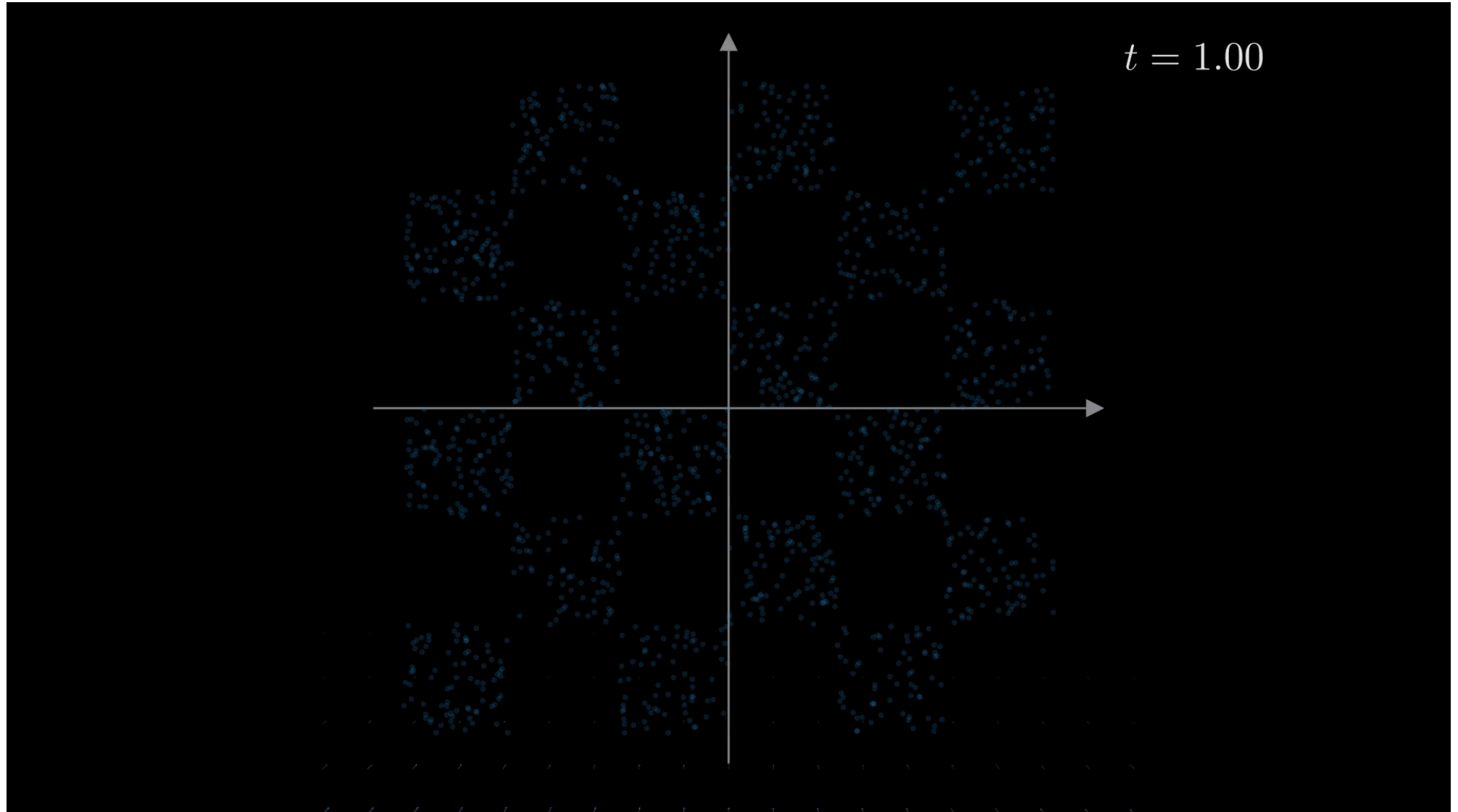
$$\hat{s} = \arg \min_s \mathbb{E}_{\mathbf{z}_0, \mathbf{y}, \epsilon, t} \left[\omega_t \left\| s(\mathbf{z}_t, \mathbf{y}, t) - \nabla_{\mathbf{z}_t} \log p(\mathbf{z}_t \mid \mathbf{z}_0) \right\|_2^2 \right]$$

- This has the **same minimizer** as regression on the marginal $\nabla_{\mathbf{z}_t} \log p(\mathbf{z}_t)$ (Song et al., 2021; Vincent, 2011).
- The neural network can be **any architecture** which predicts a score, e.g., a MLP (Sharrock et al., 2024) or a transformer (Gloeckler et al., 2024).
- Because $\mathbf{z}_t = \alpha_t \mathbf{z}_0 + \sigma_t \epsilon$ is Gaussian, the conditional target is **known in closed form**:

$$\nabla_{\mathbf{z}_t} \log p(\mathbf{z}_t \mid \mathbf{z}_0) = -\epsilon / \sigma_t$$

- Training reduces to **noise prediction**: draw $(\mathbf{z}_0, \mathbf{y})$ from simulations, $t \sim \mathcal{U}[0, 1]$, and $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- Predicting $\mathbf{z}_0$, ϵ , the velocity, or the score are equivalent parameterizations (**design choice**) of the same object.

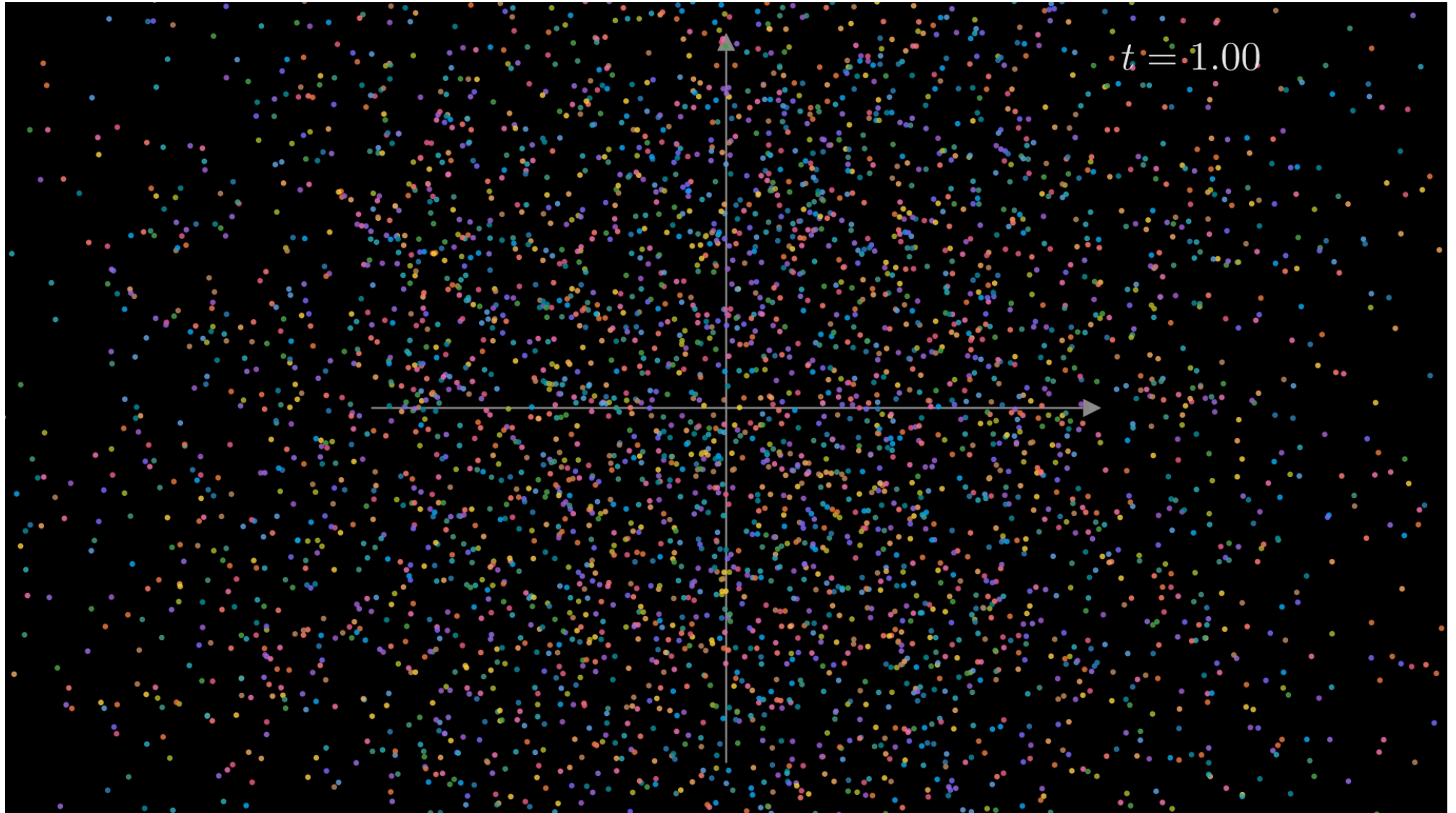
What the Learned Score Looks Like



The *trained* model's score $s(\mathbf{z}, t) \approx \nabla_{\mathbf{z}} \log p_t(\mathbf{z})$ points toward the data.

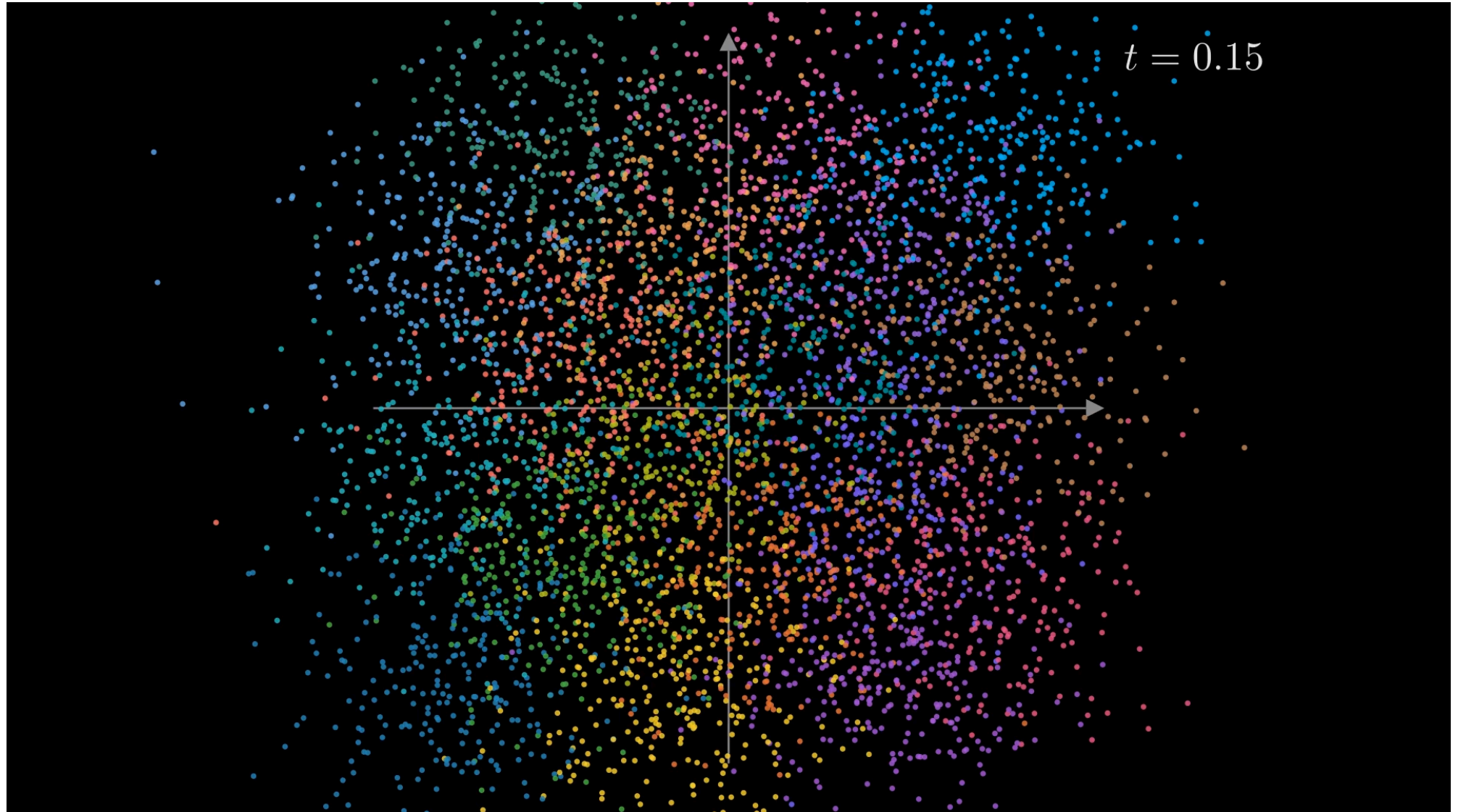
Jonas Arruda · University of Bonn · ProbAI School 2026

Reverse Process: Stochastic SDE



The **reverse SDE** injects noise at every step.

Reverse Process: Stochastic SDE Close-Up



The same trajectory, zoomed into the last time steps.

From SDE to ODE: Flow Matching

- The reverse SDE has a deterministic twin, the **probability-flow ODE** with the *same* marginals:

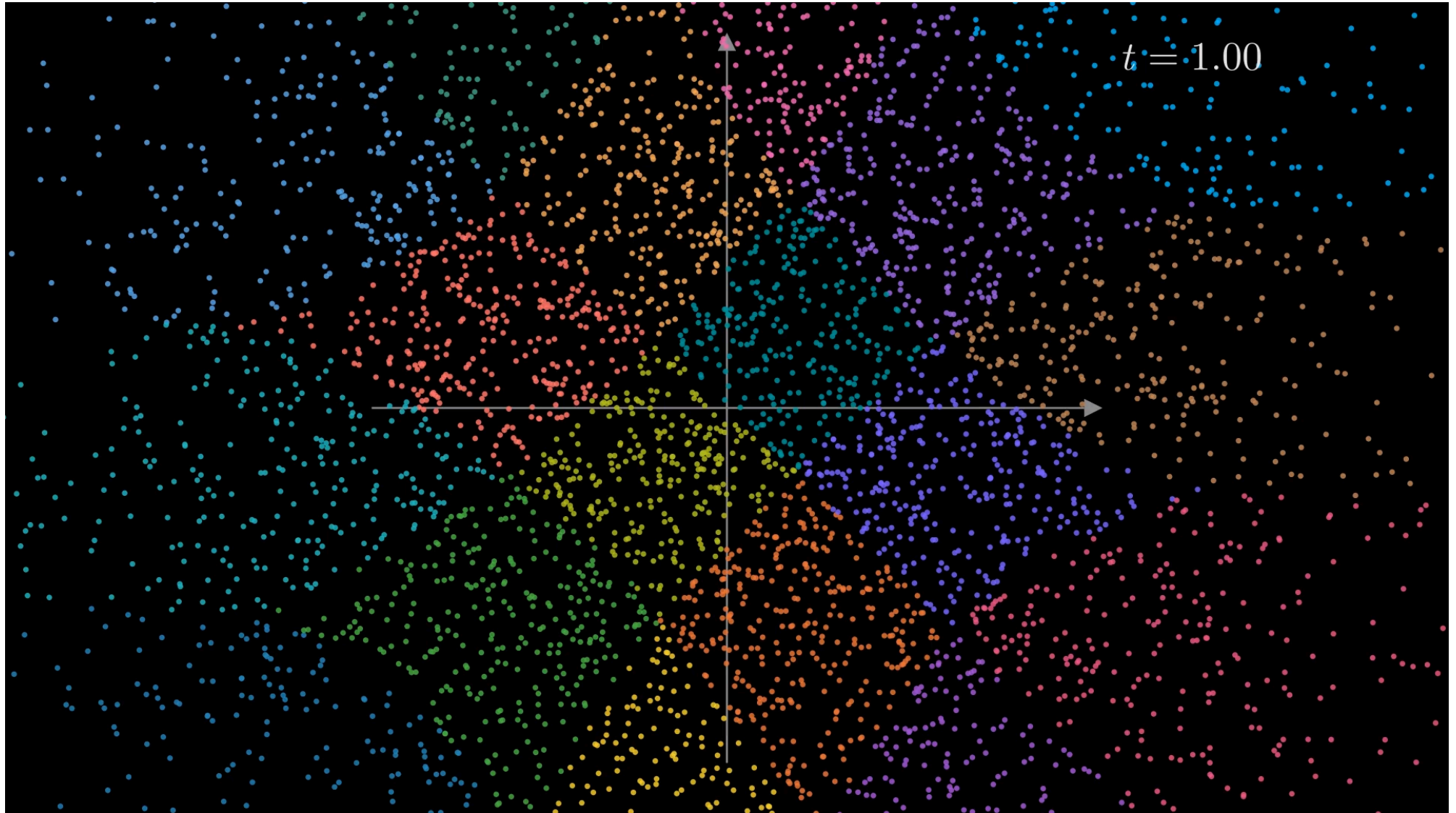
$$d\mathbf{z}_t = \left[f(t)\mathbf{z}_t - \frac{1}{2}g(t)^2 s(\mathbf{z}_t, \mathbf{y}, t) \right] dt = v(\mathbf{z}_t, \mathbf{y}, t) dt$$

- Flow matching** (Lipman et al., 2023; Wildberger et al., 2023) skips the score and regresses the **velocity field** v directly, along a chosen interpolation between noise and data:

$$\hat{v} = \arg \min_v \mathbb{E}_{t, \mathbf{z}_0, \epsilon} [\|v(\mathbf{z}_t, \mathbf{y}, t) - (\dot{\alpha}_t \mathbf{z}_0 + \dot{\sigma}_t \epsilon)\|_2^2]$$

- Straighter paths, so usually fewer integration steps.
- Density evaluation** is available by integrating the Jacobian trace along the ODE, but it is expensive.

Reverse Process: Deterministic ODE



The **probability-flow ODE** has the *same marginals* as the SDE, but a deterministic path.

Consistency Models

- Both the SDE and the ODE need many solver steps, which makes sampling slow.
- **Consistency models** (Schmitt et al., 2025; Song et al., 2023) learn a map that is **invariant along a trajectory**.
- It can jump straight to the endpoint:

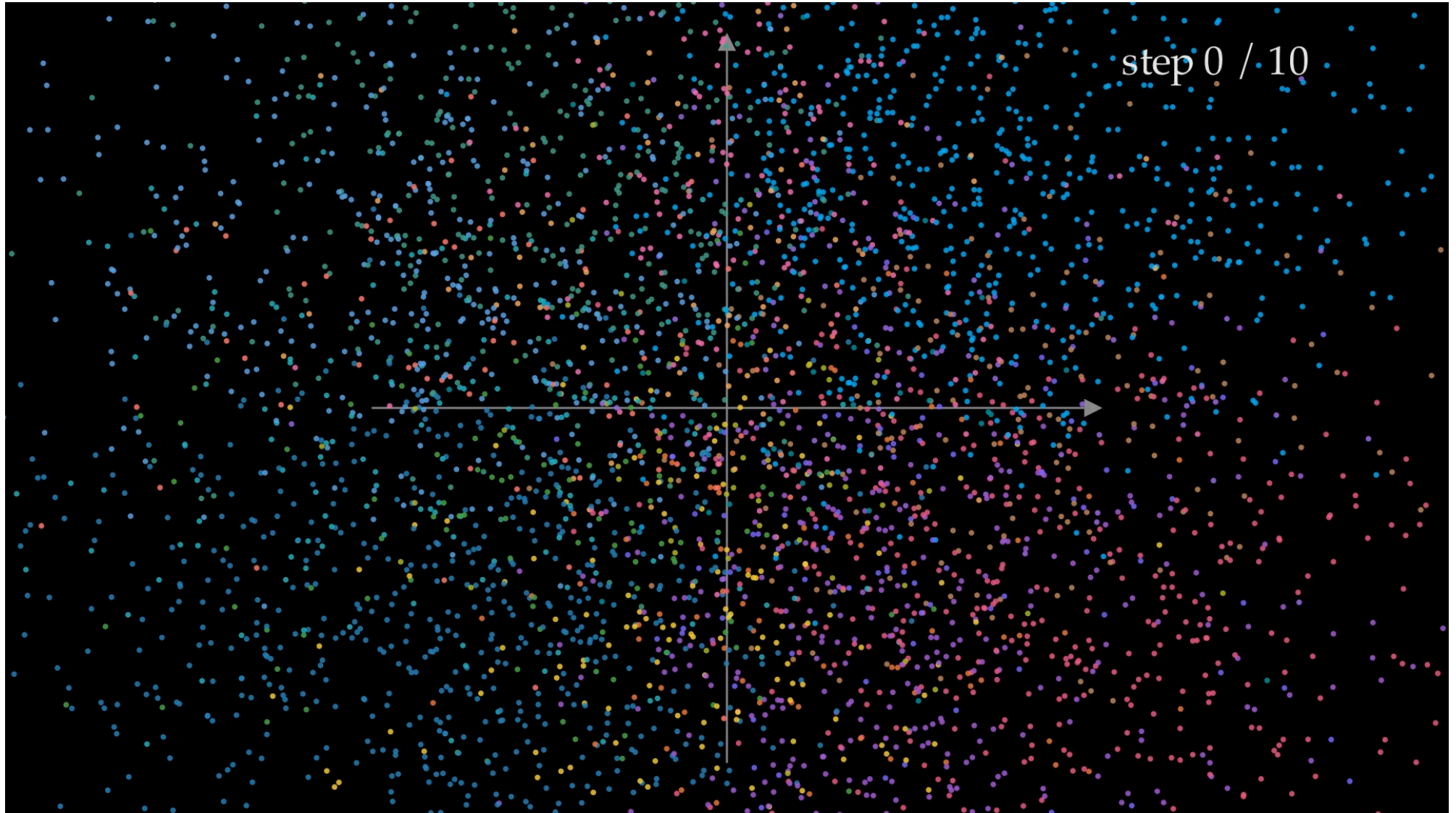
$$c(\mathbf{z}_t, \mathbf{y}, t) = c(\mathbf{z}_{t'}, \mathbf{y}, t'), \quad c(\mathbf{z}_0, \mathbf{y}, 0) = \mathbf{z}_0$$

- Parameterize $c(\mathbf{z}_t, \mathbf{y}, t) = c_{\text{skip}}(t) \mathbf{z}_t + c_{\text{out}}(t) F(\mathbf{z}_t, \mathbf{y}, t)$ with $c_{\text{skip}}(0) = 1$, $c_{\text{out}}(0) = 0$, and train outputs to **agree at nearby times**:

$$\min_c \mathbb{E} \left[\omega_t d(c(\mathbf{z}_t, \mathbf{y}, t), \bar{c}(\mathbf{z}_{t-\Delta t}, \mathbf{y}, t - \Delta t)) \right]$$

→ **one- or few-step** sampling: jump to the endpoint, re-noise to a lower level, and refine.

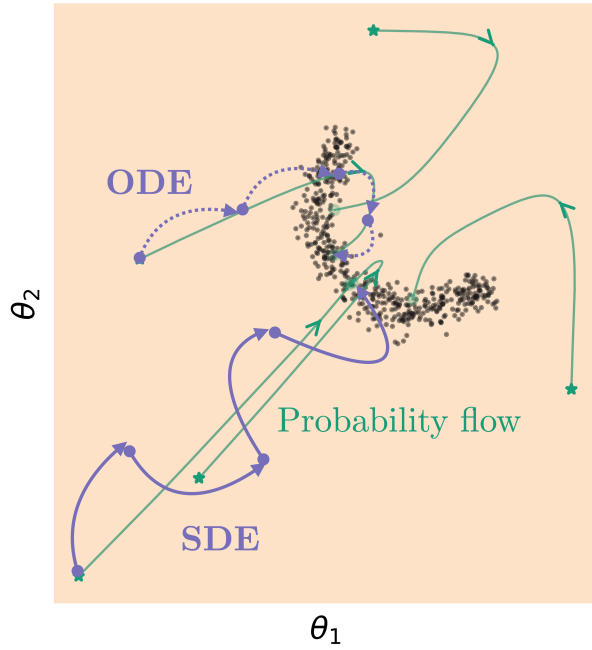
Consistency Sampling: Few Discrete Steps



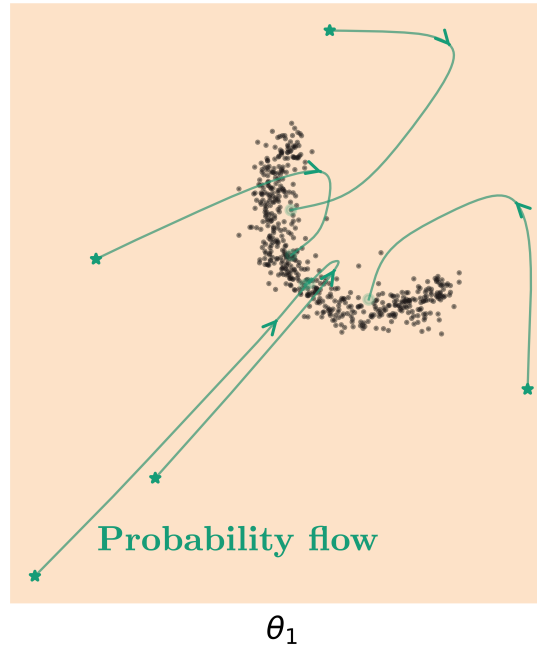
Each step **jumps straight to data**, then **re-noises** to a lower level.

The Diffusion Model Family

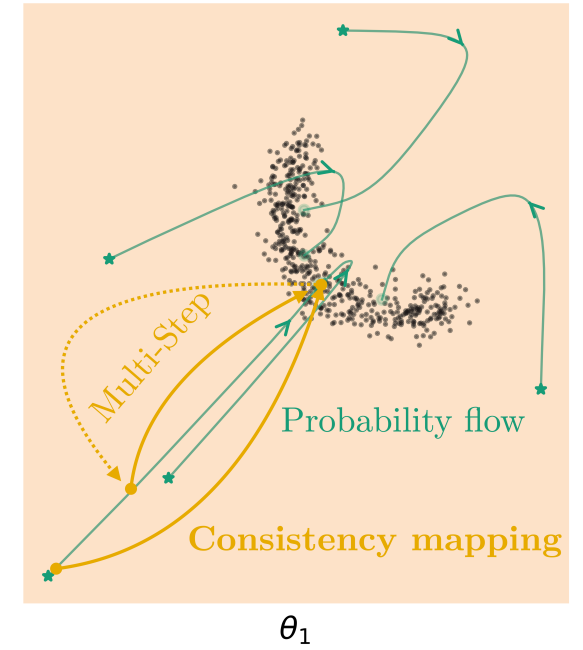
Diffusion Models



Flow Matching



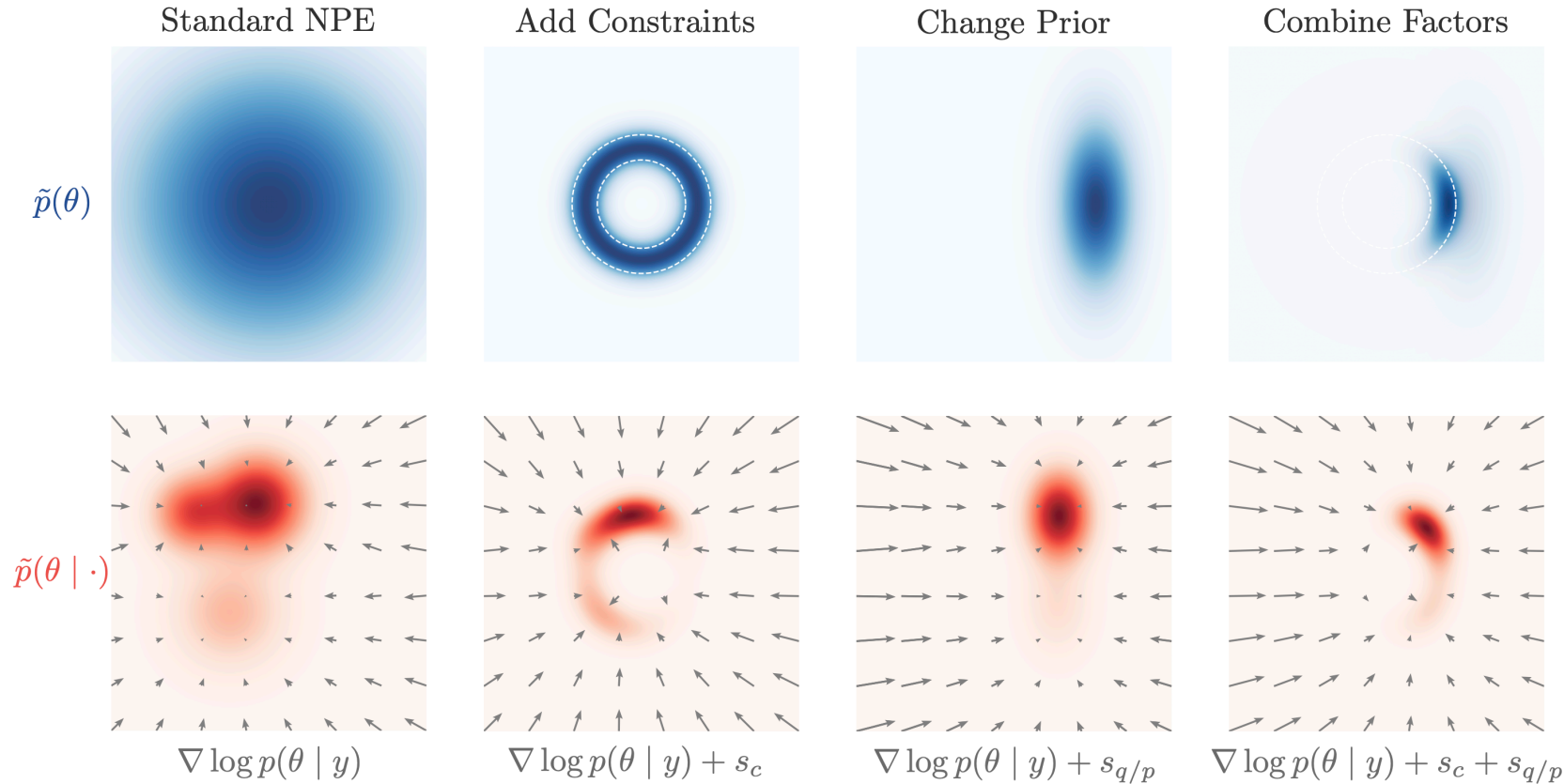
Consistency Models



Diffusion model (stochastic) · flow matching (deterministic ODE) · consistency model (direct jump).

Post-Hoc Guidance

The score is **additive**, so sampling can be steered after training by adding a gradient term.



$$\nabla_{\theta_t} \log p(\theta_t | \text{extra}) \approx \hat{s}(\theta_t, \mathbf{y}, t) + \nabla_{\theta_t} \log g(\theta_t)$$

→ Impose constraints, change the prior, or compose models without retraining (Bansal et al., 2023; Yang et al., 2026).

Careful: Guidance Changes the Target

- Guidance modifies the reverse-time marginals, so the density you sample from is generally **not** the one you intended ([Chidambaram et al., 2024](#)).
 - Samples can be **biased**, and how badly depends on the type of guidance.
 - Correction samplers (Langevin, MCMC, or weighting) can recover the target ([Geffner et al., 2023](#); [Skreta et al., 2025](#)).
- So re-check calibration *after* guidance, with the tools from Part 2.

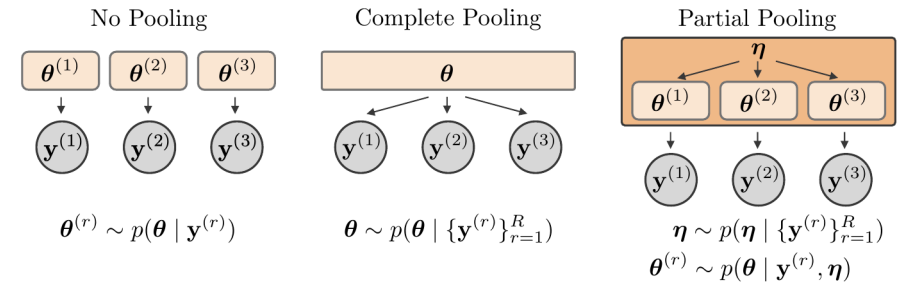
Revisiting Hierarchical Models

Data comes in **groups** (subjects, experiments, sites): local variation, shared global structure (Gelman et al., 2013).

$$\boldsymbol{\eta} \sim p(\boldsymbol{\eta}), \quad \boldsymbol{\theta}^{(r)} \sim p(\boldsymbol{\theta} \mid \boldsymbol{\eta}), \quad \mathbf{y}^{(r)} \sim p(\mathbf{y} \mid \boldsymbol{\theta}^{(r)})$$

Two coupled inference targets:

$$p(\boldsymbol{\eta} \mid \{\mathbf{y}^{(r)}\}_{r=1}^R), \quad p(\boldsymbol{\theta} \mid \mathbf{y}^{(r)}, \boldsymbol{\eta})$$



Bottleneck: simulation of a single observation is a set of sets.

Compositional Hierarchical Inference

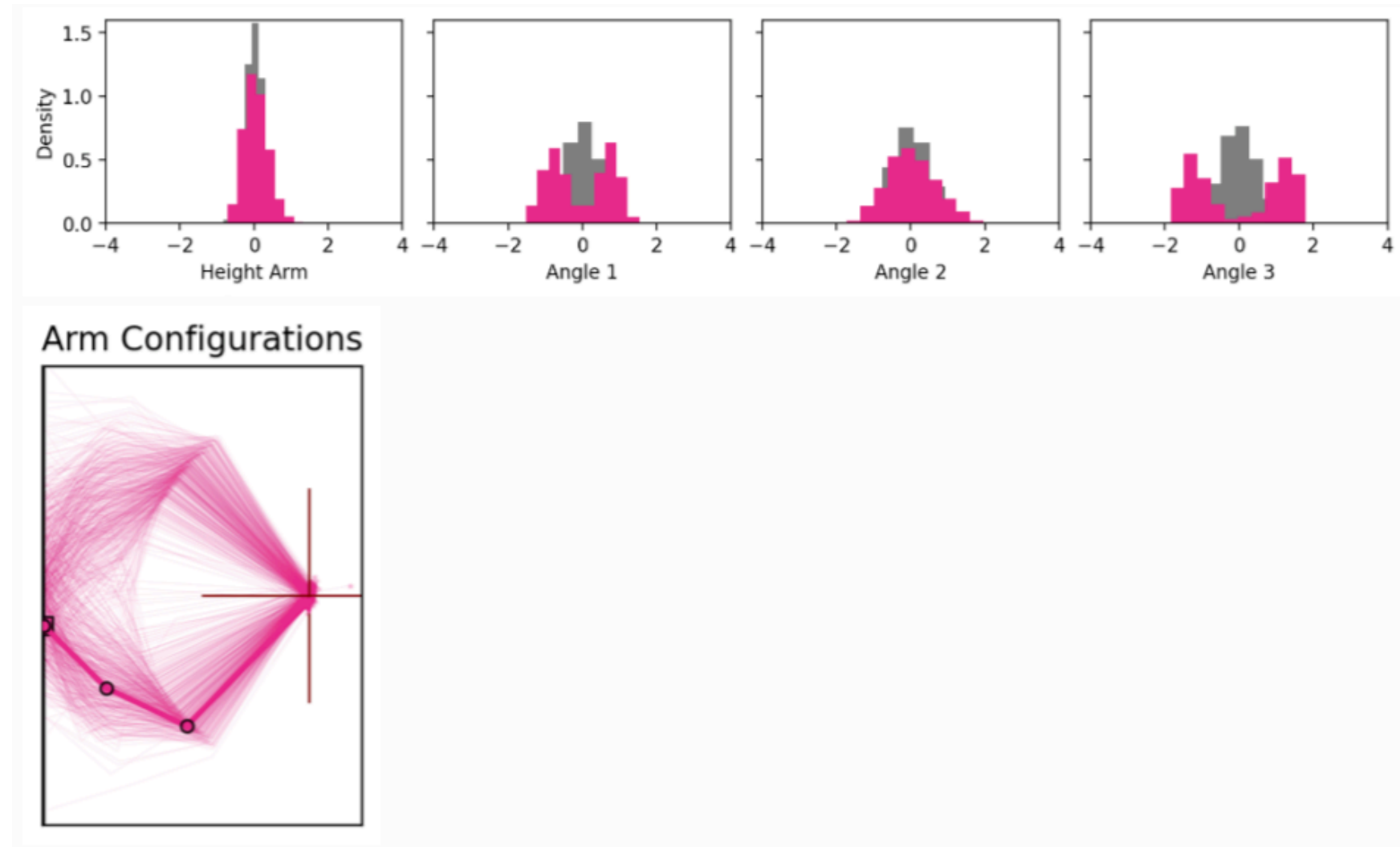
Additive scores also solve the hierarchical bottleneck Compose **per-group** posteriors instead of simulating the full hierarchy (Arruda et al., 2026):

$$\nabla_{\boldsymbol{\eta}} \log p(\boldsymbol{\eta} \mid \{\mathbf{y}^{(r)}\}) = (1 - R) \nabla_{\boldsymbol{\eta}} \log p(\boldsymbol{\eta}) + \sum_{r=1}^R \nabla_{\boldsymbol{\eta}} \log p(\boldsymbol{\eta} \mid \mathbf{y}^{(r)})$$

Train on **single groups**; at inference, **add the scores** and sample the reverse SDE. This scales to **250K+ groups**, with a total simulation budget smaller than one simulation of the full hierarchical model.

Revisiting the Robot Arm

Back to the toy example from Part 1. A diffusion model with a plain MLP score net recovers the multimodal, non-identifiable geometry:



Exercise 3

Diffusion Models & Custom Guidance

- Train a **diffusion** posterior for the multimodal **inverse-kinematics** arm.
- Compare it against a normalizing flow and a **consistency** model.
- **Write your own guidance constraint** and steer sampling after training via `guidance_kwargs`.



► Open in Colab 03_Diffusion_Guidance at <https://shorturl.at/jTU9J>

Take-Home

1. **SBI** needs nothing but a simulator you can sample.
2. **Amortization** trades a one-time training cost for instant inference on any dataset.
3. **Summary networks** encode the structure of the data (set, time series, hierarchy).
4. **Diagnostics** such as SBC for calibration or predictive checks for misspecification.
5. **Diffusion models** drop invertibility, which is what makes them **steerable after training**.

Some open questions I am thinking about currently:

- Which density do we target after post-hoc guidance?
- Can we guide a consistency model?
- What to amortize over vs. adapt post hoc?

Whatever you build: check it with simulation-based calibration!

Additional Resources

DIFFUSION MODELS IN SIMULATION-BASED INFERENCE: A TUTORIAL REVIEW

Jonas Arruda
Bonn Center for Mathematical Life Sciences,
Life & Medical Sciences Institute,
University of Bonn, Germany

Niels Bracher
Center for Modeling, Simulation, & Imaging
in Medicine (CeMSIM)
Rensselaer Polytechnic Institute, NY, USA

Ullrich Köthe
Computer Vision and Learning Lab,
Heidelberg University, Germany

Jan Hasenauer
Bonn Center for Mathematical Life Sciences,
Life & Medical Sciences Institute,
University of Bonn, Germany

Stefan T. Radev
Center for Modeling, Simulation, & Imaging in Medicine (CeMSIM)
Rensselaer Polytechnic Institute, NY, USA



Tutorial review paper: 50+ SBI & diffusion models papers, benchmarks, and discussion of design choices.

► Slides & Tutorial with Solutions: <https://github.com/arrjon/BayesFlowTutorial>

Reach out: jonas.arruda@uni-bonn.de

